Lecture 11: UDP Client-Server Programming I

Objectives:

- Learn how to create UDP applications using the **UdpClient** class
- Learn how to create UDP applications using the **Socket** class

1. Creating UDP applications using the UdpClient class

The UdpClient class:

UdpClient class is designed to simplify the process of creating UDP applications.
However, since UDP is connectionless oriented, there is no listener component.
Both server and client applications are created using the UdpClient class.

Constructor:

`UdpClient()`	Bound the new instance to any local IP and any local port
`UdpClient(int port)`	Bounds the new instance to a specific local port
`UdpClient(IPEndPoint localEP)`	Bounds the new instance to a specific local IP endpoint
`UdpClient(string host, int port)`	Bounds the new instance to any local endpoint and associated it with a default remote host and default remote port.

Methods:

`void Connect(IPEndPoint remoteEP)`
`void Connect(IPAddress ip, int port)`
`void Connect(String host, int port)`
	Establishes a default remote host and default remote port.

`public byte[] Receive(ref IPEndPoint remoteEP)`	Returns a UDP datagram sent by a remote host.

`int Send(byte[] data, int size)`
`int Send(byte[] data, int size, IPEndPoint remoteEP)`
`int Send(byte[] data, int size, string host, int port)`
	Sends a datagram to the client. The first version assumes a default remote end-point has been established.

`void Close()`	Closes the UDP connection.

Notes about the Receive method:

We do not need to create a fixed size array before calling the Receive method. The method returns an array big-enough to store the received datagram.

The Receive method uses the ***ref*** modifier to capture the IPEndPoint of the remote client.

The local client has to first create a dummy IPEndPoint and then use it to call the ***Receive*** method.

The Receive method will then replace the content of this dummy IPEndPoint with that of the remote client:

A simple UdpServer

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

class SimpleUdpServer
{
   public static void Main()
   {
      IPEndPoint localEP = new IPEndPoint(IPAddress.Any, 9050);
      UdpClient server = new UdpClient(localEP);

      Console.WriteLine("Waiting for a client...");

     //dummy IP
     IPEndPoint remoteEP =  new IPEndPoint(IPAddress.Any, 0);

      byte[] data;

      while(true)
      {
         data = server.Receive(ref remoteEP);
           Console.Write("Received from {0}: ", remoteEP.ToString());
         Console.WriteLine(Encoding.ASCII.GetString(data, 0, data.Length));
         server.Send(data, data.Length, remoteEP);
      }
   }
}
```

A simple UdpClient

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

class SimpleUdpClient {
   public static void Main()    {
      UdpClient client = new UdpClient("127.0.0.1", 9050);

      IPEndPoint remoteEP =  new IPEndPoint(IPAddress.Any, 0);

      byte[] data;
      string input;

      while(true)
      {
         Console.Write("Enter message for server or exit to stop: ");
         input = Console.ReadLine();
         if (input == "exit")
            break;

         client.Send(Encoding.ASCII.GetBytes(input), input.Length);
         data = client.Receive(ref remoteEP);
           Console.Write("Echo Received from {0}: ", remoteEP.ToString());
         Console.WriteLine(Encoding.ASCII.GetString(data, 0, data.Length));
      }
      Console.WriteLine("Stopping client");
      client.Close();
   }
}
```

Protocol

Notice that similar to TCP, there must be a protocol based on which a server and client communicate. In this case:

- The server first receives a message from client – so that it captures the client's EndPoint
- Then it echoes the message back to the client and the process is repeated.
- The reverse process takes place at the client.

Note the following points arising from the *connection-less* nature of UDP:

- There is no ***AcceptClient*** method tha returns the client's socket.
- **NetworkStream**, **StreamReader** and **StreamWriter** cannot be used for exchange of messages.
- The server can receive from any number of clients at the same time.

2. Creating UDP applications using the Socket class:

As we saw in lecture 9, the, **Socket** class also has methods for creating UDP servers and clients.

Creating a UDP Server using the Socket class

Because UDP is connectionless, only two things need to be specified to create a server application:

- Create a Socket object
- ***Bind*** the socket to a local IPEndPoint
  (No need for ***Listen*** and ***Accept*** methods)

Once the socket is created and Bound to a local end-point, the socket can be used to send and receive UDP datagrams.

However, since there is no connection, the ***Send*** and ***Receive*** methods of the socket class cannot be used.

Instead, the ***SendTo*** and ***ReceiveFrom*** methods are used. The formats of these two methods are shown below:

```
int SendTo(byte[] data, EndPoint remote);
int SendTo(byte[] data, SocketFlags flag, EndPoint remote);
int SendTo(byte[] data, int size, SocketFlags flag,
                    EndPoint remote);
int SendTo(byte[] data, int offset, int size, SocketFlags
flag, EndPoint remote);
```

```
int ReceiveFrom(byte[] data, ref EndPoint remote);
int ReceiveFrom(byte[] data, SocketFlags flag,
                              ref EndPoint remote);
int ReceiveFrom(byte[] data, int size, SocketFlags flag,
                              ref EndPoint remote);
int ReceiveFrom(byte[] data, int offset, int size,
                              SocketFlags flag, ref EndPoint
remote);
```

Example:

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

class UdpSocketServer
{
     public static void Main()
     {
          IPEndPoint localEP = new IPEndPoint(IPAddress.Any, 9050);

          Socket server = new Socket(AddressFamily.InterNetwork,
                    SocketType.Dgram, ProtocolType.Udp);

          server.Bind(localEP);
          Console.WriteLine("Waiting for a client...");

          //dummy end-point
          EndPoint remoteEP = new IPEndPoint(IPAddress.Any, 0);

          int recv;
          byte[] data;

          while(true)
          {
               data = new byte[1024];
               recv = server.ReceiveFrom(data, ref remoteEP);
               Console.Write("Received from {0}: ",
remoteEP.ToString());
               Console.WriteLine(Encoding.ASCII.GetString(data, 0,
recv));
               server.SendTo(data, recv, SocketFlags.None, remoteEP);
          }
     }
}
```

Again we note that stream communication is not possible even with the Socket class since we do not have reference to the client's Socket.

Creating a UDP client using the Socket class

Because the UDP client does not need to wait on a specific port for incoming data, it does not need to use Bind() method. Instead, it uses the random port generated by the system.

When the client sends a message to the server, the server will capture its IPEndPoint and use same to send message to it.

Example:

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

class SimpleUdpClient
{
     public static void Main()
     {
          Socket client = new Socket(AddressFamily.InterNetwork,
                                                  SocketType.Dgram,
ProtocolType.Udp);

          EndPoint remoteEP = new IPEndPoint(IPAddress.Parse("127.0.0.1"),
9050);
          //Note -- need to use EndPoint for the ReceiveFrom to work!

          byte[] data;
          string input;
          int recv;


          while(true)
          {
               Console.Write("Enter message for server or exit to stop: ");
               input = Console.ReadLine();
               if (input == "exit")
                    break;
               client.SendTo(Encoding.ASCII.GetBytes(input), remoteEP);
               data = new byte[1024];
               recv = client.ReceiveFrom(data, ref remoteEP);
               Console.Write("Echo from from {0}: ", remoteEP.ToString());
               Console.WriteLine(Encoding.ASCII.GetString(data, 0, recv));
          }
          Console.WriteLine("Stopping client");
          client.Close();
     }
}
```

Using the Connect Method with Udp client:

Because there is no connection in UDP, the ***SendTo*** and ***ReceiveFrom*** methods are used to specify the destination address.

However, if a UDP application is sending and receiving data from only one source, then the ***Connect*** method can be used to connect the application with the source. Once this is done, the ***Send*** and ***Receive*** methods can be used to send messages.

Example:

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;

class ConnectedUdpSocketClient
{
     public static void Main()
     {
          Socket client = new Socket(AddressFamily.InterNetwork,
                                                  SocketType.Dgram,
ProtocolType.Udp);

          EndPoint remoteEP = new IPEndPoint(IPAddress.Parse("127.0.0.1"),
9050);
          client.Connect(remoteEP);

          byte[] data;
          string input;
          int recv;


          while(true)
          {
               Console.Write("Enter message for server or exit to stop: ");
               input = Console.ReadLine();
               if (input == "exit")
                    break;
               client.Send(Encoding.ASCII.GetBytes(input));
               data = new byte[1024];
               recv = client.Receive(data);
               Console.Write("Echo from from {0}: ", remoteEP.ToString());
               Console.WriteLine(Encoding.ASCII.GetString(data, 0, recv));
          }
          Console.WriteLine("Stopping client");
          client.Close();
     }
}
```